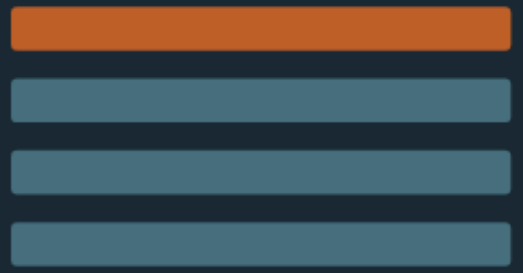
MAESTRÍA · ESPACIO ACADÉMICO

Gestión de Configuración y Mantenimiento de Software

Sesión 4 — Unidad 4 · Estrategias de mantenimiento y evolución

El sistema que dura demasiado tiene sus propios problemas.



César Augusto Vega Fernández

Encuentro sincrónico · Refactorización · Modernización · Observabilidad · Evaluación del impacto · Presentación final

EL RETO DE LA UNIDAD 4 · ACTIVIDADES EN LA PLATAFORMA

Mejorar el sistema y demostrar que mejoró

«Ya sabemos qué le pasa al sistema, ya controlamos sus cambios y ya entregamos con un pipeline. Ahora queremos que sea mejor y queremos verlo en números, no en opiniones.»

Responsable del sistema que eligió cada grupo

Actividad 4 · Propuesta e implementación de mejoras

- 25 % · 20 horas · grupal, de 2 a 3
- Documento técnico de 8 a 10 páginas o README.md
- Código o evidencias de la implementación
- Resultados de la mejora: el antes y el después

Actividad 5 · Presentación final del proyecto

- 10 % · 10 horas · grupal
- Presentación de 10 a 15 diapositivas y demo funcional
- Resumen ejecutivo de 2 a 3 páginas o README.md

La rúbrica pide mostrar la evidencia del antes y el después. Una mejora que no se midió antes no se puede demostrar después.

Los formatos de esta unidad son del F-22 al F-28. En el repositorio hay un ejercicio de refactorización con cada paso en un commit y su CI en verde.

CÓMO VAMOS A USAR LA SESIÓN

Primero qué significa, después ejemplos, después ustedes

PRIMERO

Conceptos

Del diagnóstico a las mejoras, refactorización, modernización, observabilidad y cómo se evalúa el impacto.

DESPUÉS

Ejemplos

El ejercicio de refactorización del curso, paso a paso, y la reestructuración de Flask 3.0 medida con la misma herramienta.

Y EL RESTO DE LA SESIÓN

El hacer

El taller en salas: elegir las mejoras, medir el antes y dar el primer paso con pruebas de caracterización.

El bloque que no se recorta es el tercero: la medición del antes solo se puede hacer antes.

BLOQUE 1 · EL PROBLEMA

La paradoja del software exitoso

Un sistema que nadie usa

Se puede reemplazar sin consecuencias.

Nadie se opone a cambiarlo.

Un sistema que sostiene la operación

Se vuelve intocable precisamente por su utilidad.

Cada despliegue es un evento que se planea con días de anticipación.

«Lo arreglamos después»

Bajo presión de fecha, el equipo ve un componente mal estructurado y no hay tiempo de arreglarlo. Alguien crea un ticket: «refactorizar el componente X». Un mes después hay cincuenta tickets encima; seis meses después el atajo es arquitectura permanente.

Los sistemas más exitosos son, con el tiempo, los más difíciles de mantener. Es el punto de partida de cualquier conversación sobre modernización.

BLOQUE 1 · DE DÓNDE SALEN LAS MEJORAS

Del diagnóstico a las oportunidades de mejora

Fuente	Qué aporta
F-05 · Deuda técnica (Unidad 1)	Dónde está el costo que se paga en cada cambio
F-06 · Riesgos (Unidad 1)	Qué puede salir mal y con qué impacto
F-11 · Control de cambios (Unidad 2)	Qué cambios se repiten y cuánto cuestan
F-20 · Métricas de entrega (Unidad 3)	Dónde se demora un cambio en llegar a producción

Cómo priorizar

Valor: qué riesgo o costo elimina.

Costo: cuánto trabajo toma.

Riesgo: qué puede romper.

Se eligen dos o tres mejoras que quepan en 20 horas, no diez.

La rúbrica pide identificar las mejoras de forma clara y bien justificada: cada mejora debe rastrearse hasta algo que el grupo ya encontró en el sistema.

BLOQUE 1 · REFACTORIZACIÓN

Una inversión estructural, no un costo visible

1. **Cambia la estructura, no el comportamiento**
   No agrega funcionalidad ni corrige errores: si el comportamiento cambia, ya no es refactorización.
2. **En pasos pequeños**
   Cada paso se puede verificar, revertir y explicar en un mensaje de commit.
3. **Con pruebas antes de empezar**
   Las pruebas son lo que demuestra que el comportamiento se conservó.
4. **Con el pipeline como red**
   Cada paso pasa por la CI de la Unidad 3 antes de integrarse.

Fowler (2018)

Refactorizar es mejorar el diseño con pequeñas transformaciones que preservan el comportamiento observable.

Lo que se gana es que el siguiente cambio cueste menos.

Refactorizar y corregir un error son cambios distintos y van en commits distintos: así el historial dice cuál cambió el comportamiento.

BLOQUE 1 · ANTES DE TOCAR EL CÓDIGO

Pruebas de caracterización

Prueba usual	Prueba de caracterización
Fija lo que el sistema debería hacer.	Fija lo que el sistema hace hoy, incluidas sus rarezas.
Se escribe desde la especificación.	Se escribe ejecutando el código y anotando el resultado.

Lo que pasó en el ejercicio del curso

Un envío urbano de 1,3 kg cuesta 8.450 y el módulo lo cobra 8.400, no 8.500: round() de Python lleva los .5 al número par. La prueba fija lo que el módulo hace. La rareza no se corrigió durante la refactorización: quedó registrada como deuda, para cambiarla después en un commit propio.

El término es de Feathers (2004), Working Effectively with Legacy Code.

BLOQUE 1 · MODERNIZACIÓN · HERRAMIENTA EN VIVO

¿Reemplazar o evolucionar?

Estrategia	Cuándo tiene sentido	Riesgo principal
Refactorizar en el lugar	Los problemas están localizados, hay pruebas y alguien entiende el diseño	Que no se termine si los problemas están en todas partes
Modernización incremental	La plataforma pierde soporte o un componente frena todo; las interfaces están claras	Sostener la convivencia entre lo viejo y lo nuevo
Reemplazo completo	El negocio necesita otra cosa y hay recursos para operar dos sistemas	Dejar al negocio sin red mientras se construye el nuevo

El higo estrangulador

Fowler (2004): poner una fachada delante del sistema viejo y reemplazar por detrás, una funcionalidad a la vez.

El sistema nunca deja de operar.

En el aula se usa el interactivo de reemplazar o evolucionar: ocho preguntas sobre el sistema del grupo y la justificación de cada respuesta.

BLOQUE 1 · OBSERVABILIDAD

Entender el sistema desde adentro

Métricas

Comportamiento agregado: peticiones por segundo, errores, latencia.

Registros

Eventos con contexto: qué pasó, cuándo y con qué datos.

Trazas

El recorrido de una petición entre componentes.

Los objetivos de nivel de servicio (SLO) ponen un umbral explícito a lo aceptable: por ejemplo, el 99 % de las peticiones responde en menos de 300 ms. Así se sabe si una mejora mejoró.

Beyer, Jones, Petoff y Murphy (2016), Site Reliability Engineering. Herramientas: Prometheus, Grafana, OpenTelemetry. La ruta /salud de la plantilla es el comienzo.

BLOQUE 1 · EVALUACIÓN DEL IMPACTO

Medir antes de declarar éxito

Si la mejora busca…	Mida antes y después…
Que cambiar el código cueste menos	Complejidad, duplicación, tamaño de las funciones, tiempo de un cambio típico
Menos errores en producción	Defectos reportados, tasa de fallos de cambio, SLO
Entregar más rápido	Frecuencia de despliegue y tiempo de entrega (Unidad 3)
Mejor rendimiento	Latencia y consumo bajo una carga definida
Salir de una plataforma sin soporte	Componentes migrados, dependencias sin soporte que quedan

Tres reglas

Declarar el objetivo antes de medir.

Medir el antes en una versión identificada (Unidad 2).

Una métrica sola puede engañar: miren varias.

herramientas/impacto.py compara dos versiones de un repositorio: complejidad, anidamiento, funciones largas, duplicación y número de pruebas.

BLOQUE 1 · SOSTENIBILIDAD

Una práctica, no un estado

Diagnosticar
Unidad 1 · qué le pasa al sistema

›

Controlar
Unidad 2 · qué cambia y quién lo aprueba

›

Automatizar
Unidad 3 · cada cambio verificado

›

Mejorar
Unidad 4 · con red de seguridad

›

Medir
y volver a diagnosticar

La norma ISO/IEC/IEEE 14764 trata la mantenibilidad como algo que se planea desde el inicio del ciclo de vida. Los sistemas se vuelven sostenibles por la suma de decisiones cotidianas bien fundamentadas.

BLOQUE 2 · EJEMPLO GUIADO 1 · F-22 Y F-24

El ejercicio: un módulo que nadie quiere tocar

El código heredado

Una función de 41 líneas con tres bloques casi idénticos, uno por zona de envío.

Complejidad 15, anidamiento 5.

Cambiar una tarifa exige tocar tres lugares sin equivocarse.

Antes de mover nada

24 pruebas de caracterización: tarifas por zona y peso, envío express, descuentos, redondeo, zona desconocida y factura.

Una de ellas fija la rareza del redondeo, registrada como deuda.

Carpeta ejercicio/ del repositorio. La CI corre las pruebas en cada commit y escribe el impacto del paso en el resumen de la ejecución.

Cuatro pasos, cuatro commits, cuatro ejecuciones en verde

Paso	Commit	Complejidad máxima	Anidamiento	Líneas duplicadas
Código heredado	5e6bf7c	15	5	8
1 · Extraer el cálculo por peso	06216a6	9	4	6
2 · Tabla de tarifas en lugar de condicionales	8f1e225	5	2	0
3 · Extraer el descuento y el redondeo	d51f577	4	1	0
4 · Eliminar la duplicación de la factura	3f2ee8f	3	1	0

En cada paso pasan las mismas 24 pruebas. El interactivo de refactorización muestra el código de cada commit con sus métricas.

El impacto total, y lo que una métrica sola esconde

Métrica	Antes	Después
Complejidad máxima	15	3
Complejidad promedio	9,5	2,1
Anidamiento máximo	5	1
Líneas duplicadas	8	0
Funciones de más de 25 líneas	1	0
Funciones	2	7
Casos de prueba	24	24

Lo que esconde

En el paso 3 la complejidad total subió de 12 a 14: hay más funciones, cada una simple.

Las líneas de código no bajan en el último paso: separar en funciones con nombre agrega líneas.

Lo que importa es el objetivo: cambiar una tarifa es ahora cambiar una fila.

```
python herramientas/impacto.py . --antes 5e6bf7c --despues 3f2ee8f --ruta ejercicio/envios --pruebas ejercicio/tests
```

La reestructuración de Flask 3.0, medida

src/flask, de 2.3.3 a 3.0.0	Antes	Después
Archivos	22	24
Complejidad máxima	30	23
Funciones con complejidad mayor que 10	10	10
Líneas duplicadas	477	775
Funciones de prueba	368	369

Cómo leerlo

El objetivo era separar el núcleo de la entrada y salida (Sans-IO), no reducir la complejidad.

La duplicación sube porque app.py y el nuevo sansio/app.py comparten definiciones copiadas, como _make_timedelta y las variables de tipos.

Sin el objetivo declarado, estas métricas dirían que empeoró.

python herramientas/impacto.py flask --antes 2.3.3 --despues 3.0.0 --ruta src/flask --pruebas tests

La presentación final cuenta el proyecto entero

1. **Problema**
   El diagnóstico de la Unidad 1: deuda, riesgos y evidencia.
2. **Solución**
   La estrategia de configuración (Unidad 2) y las mejoras elegidas (Unidad 4).
3. **Implementación**
   El repositorio, el pipeline y los pasos de la mejora, con sus enlaces.
4. **Resultados**
   El antes y el después, medidos, y lo que falta.

La demo

Un cambio pequeño que entra por PR, pasa la CI y llega a una versión.

Es más convincente que diez diapositivas sobre el pipeline.

Ensáyenla: un pipeline lento en vivo se ve peor que una ejecución grabada.

EL HACER · TALLER EN SALAS

De la sesión salen con el antes medido

1	**Elegir dos o tres mejoras** Desde el F-05, el F-06 y el F-20, con valor, costo y riesgo.	8 min
2	**Declarar el objetivo de cada una** Qué se espera que cambie y cómo se va a medir.	5 min
3	**Medir el antes** En una versión identificada, con impacto.py u otras métricas.	7 min
4	**Escribir las primeras pruebas de caracterización** Sobre el código que se va a tocar primero.	7 min
5	**Dar el primer paso** Un commit pequeño, por PR, con la CI en verde.	3 min

Al salir de la sala: el F-22 y el F-25 en borrador, y el primer PR abierto.

PARA TENER AL LADO DURANTE EL TALLER

Comandos y preguntas

Comandos

git tag -a antes-de-mejoras **-m** "Línea base de medición"

python herramientas/impacto.py sistema **--antes** antes-de-mejoras **--despues** HEAD **--ruta** src

pytest antes y después de cada paso

git revert si un paso salió mal

Preguntas antes de cada paso

¿Este paso cambia el comportamiento? Si sí, no es refactorización.

¿Hay una prueba que lo demuestre?

¿Cabe en un commit con un mensaje claro?

¿Qué métrica debería moverse?

CIERRE DEL CURSO

Un sistema, cuatro unidades, una práctica

ESTA SEMANA

Actividad 4

Implementar las mejoras con su antes y después medido. Verificar con el F-28 antes de entregar.

AL CIERRE

Actividad 5

Presentar el proyecto completo: problema, solución, implementación y resultados, con una demo.

La evolución del software no depende solo de escribir código: depende de tener mecanismos para gestionar el cambio de forma controlada, trazable y sostenible.

REFERENCIAS

Fuentes de la sesión

Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). Site reliability engineering: How Google runs production systems. O'Reilly Media.

Feathers, M. (2004). Working effectively with legacy code. Prentice Hall.

Fowler, M. (2004). Strangler fig application. https://martinfowler.com/bliki/StranglerFigApplication.html

Fowler, M. (2018). Refactoring: Improving the design of existing code (2.ª ed.). Addison-Wesley.

ISO/IEC/IEEE. (2022). ISO/IEC/IEEE 14764:2022 Software engineering — Software life cycle processes — Maintenance.

Parnas, D. L. (1994). Software aging. En Proceedings of the 16th International Conference on Software Engineering (pp. 279–287). IEEE.

Pallets. (2026). Flask [Repositorio de código fuente]. https://github.com/pallets/flask